6. Gyakorlat

Python kurzus

OOP II.

```
def reciprocal(n):
    try:
        n = 1 / n
    except ZeroDivisionError:
        print("Division failed")
        n = None
    else:
        print("Fine")
    finally:
        print("Goodbye")
    return n
print(reciprocal(2))
print(reciprocal(0))
```

```
Fine
Goodbye
0.5
Division failed
None
```

```
Fine
Goodbye
0.5
Division failed
Goodbye
None
```

```
Fine
Goodbye
0.5
Division failed
Goodbye
```

1. feladat: Dog osztály és leszármazottai, DogShelter osztály

a) Készíts egy Dog nevű osztályt, ami tárolja a kutya nevét, korát. A kor csak pozitív egész szám lehet, ha nem az, dobj ValueError kivételt! A kutya adatait a __str__ metódussal írasd ki.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        if not isinstance(age, int) or age <= 0:
            raise ValueError("A kor csak pozitív egész szám lehet!")
        self.age = age
    def __str__(self):
        return f"{self.name}, {self.age} éves"
```

b) Definiáld a Dog osztály leszármazott osztályait: Sheepdog, GuardDog, HuntingDog. A breed (fajta) tulajdonságot a leszármazott osztályokban tárold!
Tedd lehetővé, hogy a Sheepdog, GuardDog és HuntingDog osztályok példányai kiírathatóak legyenek a print utasítással: jelenjen meg a nevük, a koruk és a fajtájuk.

```
class Sheepdog(Dog):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.breed = "Juhászkutya"
    def __str__(self):
        return f"{self.name}, {self.age} éves, {self.breed}"

class GuardDog(Dog):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.breed = "Őrkutya"
    def __str__(self):
        return f"{self.name}, {self.age} éves, {self.breed}"

class HuntingDog(Dog):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.breed = "Vadászkutya"
    def __str__(self):
        return f"{self.name}, {self.age} éves, {self.breed}"
```

c) Készíts példányokat mind a 4 osztályból és tesztelj!

```
try:
    dog1 = Sheepdog("Bodri", 5)
    dog2 = GuardDog("Kormi", 4)
    dog3 = HuntingDog("Morzsi", 3)
    dog4 = Dog("Rex", 2)
except ValueError as e:
    print(e)
else:
    print(dog1)
    print(dog2)
    print(dog3)
    print(dog4)
```

```
Bodri, 5 éves, Juhászkutya
Kormi, 4 éves, Őrkutya
Morzsi, 3 éves, Vadászkutya
Rex, 2 éves
```

d) Készíts egy DogShelter nevű osztályt, ami tárolja a kutyákat. Egy DogShelter példány tárolja osztály szinten és példány szinten is a kutyákat a nyilvántartás miatt. Hozzáadáskor bizonyosodj meg isinstance-cel, hogy valóban kutyákat tárolunk a DogShelterben. Kivételekkel kezeld le az esetleges hibákat!

```
class DogShelter:
    overall_dogs = []          #osztályváltozó

    def __init__(self):
        self.dogs = []

    def add_dog(self, dog):
        if not isinstance(dog, Dog):
            raise TypeError("Csak Dog típusú objektumok adhatók hozzá a menhelyhez!")
        self.dogs.append(dog)
        DogShelter.overall_dogs.append(dog)          #osztályszintű lista frissítése
```

e) Biztosíts lehetőséget arra, hogy a len függvény egy adott példányban elhelyezett kutyák számát adja vissza.
f) A kutyatár kiíratásakor a kutyák életkor szerint növekvő sorrendben jelenjenek meg.

```
def __len__(self):
        return f'A menhelyen {len(self.dogs)} kutya van.'

def __str__(self):
    sorted_dogs = sorted(self.dogs, key=lambda d: d.age)
    return "\n".join(str(dog) for dog in sorted_dogs)
```

e) Készíts példányokat a DogShelter osztályból és tesztelj. Az osztályváltozó értékét is írasd ki!

```
shelter1 = DogShelter()
try:
    shelter1.add_dog(dog1)
    shelter1.add_dog(dog2)
    shelter1.add_dog(dog4)
    #shelter1.add_dog("Nem kutya")

except TypeError as e:
    print(e)
print(shelter1)
print(shelter1.__len__())

shelter2 = DogShelter()
dog5 = Sheepdog("Rex", 6)
shelter2.add_dog(dog5)
print(shelter2)
print(shelter2.__len__())
```

```
Rex, 2 éves
Kormi, 4 éves, Őrkutya
Bodri, 5 éves, Juhászkutya
A menhelyen 3 kutya van.
```

```
Rex, 6 éves, Juhászkutya
A menhelyen 1 kutya van.
```

```
# Osztályszintű lista elérése

print(DogShelter.overall_dogs)

for dog in DogShelter.overall_dogs:
    print(dog)
```

```
[<__main__.Sheepdog object at 0x0000020B77A37970>,
<__main__.GuardDog object at 0x0000020B77A37910>,
<__main__.Dog object at 0x0000020B77A37850>,
<__main__.Sheepdog object at 0x0000020B77A37730>]

Bodri, 5 éves, Juhászkutya
Kormi, 4 éves, Őrkutya
Rex, 2 éves
Lassie, 6 éves, Juhászkutya
```

Többszörös öröklődés

A többszörös öröklődés lehetővé teszi, hogy egy osztály több szülőosztályból örököljön metódusokat és attribútumokat.

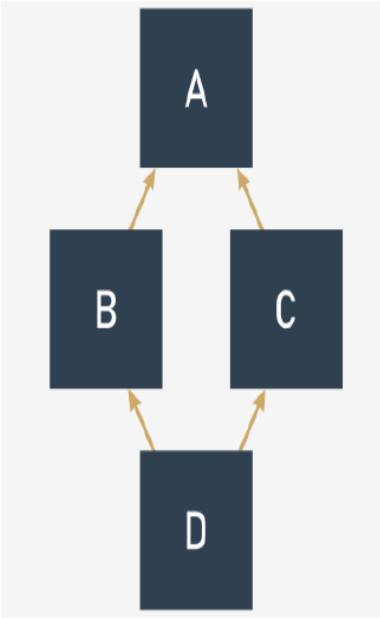
Method Resolution Order (MRO):

- Az MRO meghatározza, hogy Python milyen sorrendben keresi a metódusokat, amikor több szülőosztályból öröklünk.
- A Python a **balról jobbra** történő keresést követi.
- Az MRO sorrendje lekérdezhető az **Osztálynév.__mro__** kifejezéssel.
- **A super()** függvény az MRO alapján működik, és az MRO sorrendjének megfelelően hívja meg a szülőosztály metódusait.

Konfliktusok a többszörös öröklődésnél

Előfordulhat, hogy **ugyanaz a metódus** több szülőosztályban is létezik. Ekkor, ha egy metódust hívunk egy gyermekosztályból, a Python az MRO alapján dönti el, hogy melyik szülőosztály metódusát kell használnia.

Diamond probléma:




```
class A:
   def függvény(self):
      return "A-ból"
class B(A):
   def függvény(self):
      return "B-ből"
class C(A):
   def függvény(self):
      return "C-ből"
class D(B, C):
   pass
d = D()
```

```
print(d.függvény())        # Melyik metódus fut le
print()
print(D.__mro__)     # A szülő osztályok MRO sorrendje
```

```
B-ből


(<class '__main__.D'>, <class '__main__.B'>, <class
'__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

A super() függvény alkalmazása

A **super()** függvény alkalmazásakor közvetlenül nem kell megnevezni a szülőosztályt. Pythonban a super() az MRO alapján hívja meg a megfelelő szülőosztály metódusát.

```
class A:
  def függvény(self):
    return "A-ból"
class B(A):
  def függvény(self):
    return "B-ből " + super().függvény()
class C(A):
  def függvény(self):
    return "C-ből " + super().függvény()
class D(B, C):
  def függvény(self):
    return "D-ből " + super().függvény()
d = D()
print(d.függvény())
```

```
D-ből B-ből C-ből A-ból
```

- A super() minden osztályban meghívja a szülőosztály metódusát az MRO sorrend szerint.
- A D osztályból indulva a B osztály metódusa fut le először, majd a C osztályé, végül az A osztályé.

Példa: A super() paraméter nélkül vagy paraméterrel

```
class A:
    def greet(self):
        print("Hello from A")
class B(A):
    def greet(self):
        print("Hello from B")
class C(B):
    def greet(self):
        super().greet()                # Az aktuális osztály szuperosztályára mutat

        #super(B, self).greet()        # Explicit módon az A osztály greet metódusát hívja meg

        print("Hello from C")
c = C()
c.greet()
```

Paraméter nélkül: **super()**

```
Hello from B
Hello from C
```

Paraméterekkel: **super(cls, obj)**

```
Hello from A
Hello from C
```

Instance variables – példány változók

A Python objektumoknak előre definiált tulajdonságai és metódusai vannak. Ilyen pl. a **__dict__** metódus, amely az objektum változóinak könyvtára.

```
class ExampleClass:
    def __init__(self, val = 1):
        self.a = val
        self.__b = val * 2
example_object = ExampleClass()
print(example_object.a)
print(example_object.__b)
```

```
1
AttributeError: 'ExampleClass' object has
no attribute ,__b'
```

```
class ExampleClass:
    def __init__(self, val):
        self.a = val
        self.__b = val
example_object = ExampleClass(42)
example_object.c = 33
print(example_object.__dict__)
```

```
{'a': 42, '_ExampleClass__b': 42, 'c': 33}
```

```
class ExampleClass:
    def  __init__(self, val = 1):
        self.a = val
    def set_b(self, val):
        self.__b = val
example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_2.set_b(3)
example_object_3 = ExampleClass(4)
example_object_3.c = 5
print(example_object_1.__dict__)
print(example_object_2.__dict__)
print(example_object_3.__dict__)
```

```
{'a': 1}
{'a': 2, '_ExampleClass__b': 3}
{'a': 4, 'c': 5}
```

Példák: privát változó kiírása kívülről, name mangling

```
class ExampleClass:

    def __init__(self, val = 1):
        self.__a = val
        self.__b = val * 2
    def get_vars(self):
        return self.__a, self.__b

example_object = ExampleClass()

print(example_object.get_vars())
print(example_object.__dict__)
print(example_object._ExampleClass__a,
        example_object._ExampleClass__b)
```

```
(1, 2)
{'_ExampleClass__a': 1, '_ExampleClass__b': 2}
1 2
```

```
class ExampleClass:

    def __init__(self, val = 1):
        self.__a = val
        self.__b = val * 2
    def set_b(self, val):
        self.__b = val
    def get_vars(self):
        return self.__a, self.__b

example_object = ExampleClass(2)

example_object.set_b(3)

print(example_object.get_vars())
print(example_object.__dict__)
```

Class variables – osztályváltozók

```
class ExampleClass:
    counter = 0                                  # Class variable
    def __init__(self, val = 1):
        self.__a = val
        ExampleClass.counter += 1
example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_3 = ExampleClass(4)
print(example_object_1.__dict__, example_object_1.counter)
print(example_object_2.__dict__, example_object_2.counter)
print(example_object_3.__dict__, example_object_3.counter)
```

```
class ExampleClass:
    __counter = 0                    # Az osztályváltozó privát
    def __init__(self, val = 1):
        self.__a = val
        ExampleClass.__counter += 1
example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_3 = ExampleClass(4)
object has no attribute 'counter'
print(example_object_1.__dict__, example_object_1._ExampleClass__counter)
print(example_object_2.__dict__, example_object_2._ExampleClass__counter)
print(example_object_3.__dict__, example_object_3._ExampleClass__counter)
```

```
{'_ExampleClass__a': 1} 3
{'_ExampleClass__a': 2} 3
{'_ExampleClass__a': 4} 3
```

Osztály dictionary példa

```
class ExampleClass:
    varia = 1                                   # Class variable
    def __init__(self, val):
        self.varia = val
        ExampleClass.varia = val
print(ExampleClass.__dict__)                    # Class dictionary
print()
example_object = ExampleClass(2)
print(ExampleClass.__dict__)                    # Class dictionary
print()
print(example_object.__dict__)                  # object dictionary
```

```
{'__module__': '__main__', 'varia': 1, '__init__': <function ExampleClass.__init__ at
0x000002ADD3663F40>, '__dict__': <attribute '__dict__' of 'ExampleClass' objects>, '__weakref__':
<attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}

{'__module__': '__main__', 'varia': 2, '__init__': <function ExampleClass.__init__ at
0x000002ADD3663F40>, '__dict__': <attribute '__dict__' of 'ExampleClass' objects>, '__weakref__':
<attribute '__weakref__' of 'ExampleClass' objects>, '__doc__': None}

{'varia': 2}
```

Attribútum ellenőrzése: a hasattr() függvény

```
class ExampleClass:
    def __init__(self, val):
        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 1
example_object = ExampleClass(1)
print(example_object.a)
print(example_object.b)
```

```
1
AttributeError: 'ExampleClass' object has no attribute 'b'
```

```
try:
    print(example_object.b)
except AttributeError:
    pass
```

```
class ExampleClass:
   attr = 1
print(hasattr(ExampleClass, 'attr'))
print(hasattr(ExampleClass, 'prop'))
```

```
True
False
```

```
if hasattr(example_object, 'b'):
    print(example_object.b)
else: print('('Nincs ilyen attribútum')
```

```
Nincs ilyen attribútum
```

Metódusok

```
class Classy:
    def other(self):
        print("other")
    def method(self):
        print("method")
        self.other()
obj = Classy()
obj.method()
```

```
class Classy:
    def method(self, par):
        print("method:", par)
obj = Classy()
obj.method(1)
obj.method([2, 3])
obj.method("Main")
```

```
class Classy:
    def visible(self):
        print("visible")
    def __hidden(self):
        print("hidden")
object = Classy()
object.visible()
try:
    object.__hidden()
except:
    print("failed")
object._Classy__hidden()
```

---

1. __init__

A konstruktor:

- nem ad vissza értéket
- nem hívható direkt sem az osztályból, sem egy objektumból
- **obj_1= Classy() # TypeError**

```
class Classy:
    def __init__(self, value):
        self.var = value
obj_1 = Classy("object")
print(obj_1.var)
```

```
object
```

2. __dict__

Python osztály és objektum eleve rendelkezik attribútumokkal, amelyeket a __dict__ jelenít meg.

```
class Classy:
    varia = 1
    def __init__(self):
        self.var = 2
    def method(self): pass
    def __hidden(self): pass
obj = Classy()
print(obj.__dict__)
print(Classy.__dict__)
```

```
{'var': 2}
{'__module__': '__main__', 'varia': 1, '__init__': <function Classy.__init__ at 0x00000138DE133D00>, 'method': <function Classy.method at 0x00000138DE133D90>, '_Classy__hidden': <function Classy.__hidden at 0x00000138DE133E20>, '__dict__': <attribute '__dict__' of 'Classy' objects>, '__weakref__': <attribute '__weakref__' of 'Classy' objects>, '__doc__': None}
```

3. __name__

Az osztály neve.

```
class Classy:
    pass
print(Classy.__name__)
obj = Classy()
print(type(obj).__name__)
```

```
Classy
Classy
```

4. __module__

Az osztályt tartalmazó modul neve.

```
class Classy:
    pass
print(Classy.__module__)
obj = Classy()
print(obj.__module__)
print(__name__ == "__main__")
```

```
__main__
__main__
True
```

__main__ valójában nem egy modul, hanem az éppen futó folyamat.

5. __bases__

```
class Super_A: pass
class Super_B: pass
class Sub(Super_A, Super_B): pass
def printBases(cls):
    print('( ', end='')
    for x in cls.__bases__:
        print(x.__name__, end=' ')
    print(')')
printBases(Super_A)
printBases(Super_B)
printBases(Sub)
print(Sub.__bases__)
print(Sub.__bases__[0].__name__)
print(Sub.__bases__[1].__name__)
```

```
( object )
( object )
( Super_A Super_B )
(<class '__main__.Super_A'>, <class '__main__.Super_B'>)
Super_A
Super_B
```

A getattr(), setattr() függvények példa

```
class MyClass: pass
obj = MyClass()
obj.a = 1
obj.b = 2
obj.i = 3
obj.ireal = 3.5
obj.integer = 4
obj.z = 5
def Inc_Int_sl(obj):
    for name in obj.__dict__.keys():
        if name.startswith('i'):
            val = getattr(obj, name)
            if isinstance(val, int):
                setattr(obj, name, val + 1)
print(obj.__dict__)
Inc_Int_sl(obj)
print(obj.__dict__)
```

Az issubclass(), isinstance() függvények példa

```
class Dog:
    kennel = 0
    def __init__(self,breed):
        self.breed = breed
        Dog.kennel += 1
    def __str__(self):
        return self.breed + " says: Woof!"
class SheepDog(Dog):
    def __str__(self):
        return super().__str__() + " Don't run away, Little Lamb!"
class GuardDog(Dog):
    def __str__(self):
        return super().__str__() + " Stay where you are, Mister Intruder!"
Rocky = SheepDog("Collie")
Luna = GuardDog ("Dobermann")
Lassie = SheepDog("Rough Collie")

print(Rocky)
print(Luna)
print(issubclass(SheepDog, Dog), issubclass(SheepDog, GuardDog))
print(isinstance(Rocky, GuardDog ), isinstance(Luna, GuardDog))
print(Luna is Luna, Rocky is Luna)
print(Rocky.kennel)
```

2. feladat: A Book osztály létrehozása

Az osztály tartalmazza egy könyv címét, íróját, oldalszámát.
Kivételekkel kezeljük le, ha ezek közül hiányzik valami a példányosításkor, és ha az oldalszám nem pozitív egész szám.
Definiáljuk a minimum összehasonlító műveleteket (<, ==, >) az oldalszám szerinti rendezéshez, és kivételt adunk, ha a másik objektum nem Book típusú.
A könyv adatait a __str__ metódussal írjuk ki.

```
class Book:
    def __init__(self, title=None, author=None, pages=None):
        if not title or not author or not pages:
            raise ValueError("Minden tulajdonság megadása kötelező!")
        if not isinstance(pages, int) or pages <= 0:
            raise ValueError("Az oldalszámnak pozitív egész számnak kell lennie!")
        self.title = title
        self.author = author
        self.pages = pages
```

```
    def __lt__(self, other):                    #less than
        if not isinstance(other, Book):
            raise Exception("Összehasonlíthatatlan adatok!")
        return self.pages < other.pages
    def __eq__(self, other):                    #equal
        if not isinstance(other, Book):
            raise Exception("Összehasonlíthatatlan adatok!")
        return self.pages == other.pages
    def __gt__(self, other):                    #greater than
        if not isinstance(other, Book):
            raise Exception("Összehasonlíthatatlan adatok!")
        return self.pages > other.pages
    def __str__(self):
        return f"{self.title} - {self.author} ({self.pages} oldal)"
```

```
book1 = Book("Harry Potter", "J.K. Rowling", 300)
book2 = Book("The Lord of the Rings", "J.R.R. Tolkien", 500)
book3 = ("The Hobbit", "J.R.R. Tolkien", 250)
print(book1)
print(book1 < book2)
print(book1 == book2)
print(book2 > book3)
```

```
Harry Potter - J.K. Rowling (300 oldal)
True
False
```

Book osztály és tesztelése

```
class Book:
  def __init__(self, title=None, author=None, pages=None):
    if not title or not author or not pages:
      raise ValueError("Minden tulajdonság megadása kötelező!")
    if not isinstance(pages, int) or pages <= 0:
      raise ValueError("Az oldalszámnak pozitív egész számnak kell lennie!")
    self.title = title
    self.author = author
    self.pages = pages
```

```
  def __lt__(self, other):                    #less than
    if not isinstance(other, Book):
      raise Exception("Összehasonlíthatatlan adatok!")
    return self.pages < other.pages
  def __eq__(self, other):                    #equal
    if not isinstance(other, Book):
      raise Exception("Összehasonlíthatatlan adatok!")
    return self.pages == other.pages
  def __gt__(self, other):                    #greater than
    if not isinstance(other, Book):
      raise Exception("Összehasonlíthatatlan adatok!")
    return self.pages > other.pages
  def __str__(self):
    return f"{self.title} - {self.author} ({self.pages} oldal)"
```

```
book1 = Book("Harry Potter", "J.K. Rowling", 300)
book2 = Book("The Lord of the Rings", "J.R.R. Tolkien", 500)
book3 = ("The Hobbit", "J.R.R. Tolkien", 250)
book4 = Book("The Hobbit", "J.R.R. Tolkien", 250)
print(book1)
print(book1 < book2)
print(book1 == book2)
try:
  print(book2 > book3)
except Exception as e:
  print(e)
```

```
Harry Potter - J.K. Rowling (300 oldal)
True
False
Összehasonlíthatatlan adatok!
```

3. feladat: Library osztály létrehozása

Készítsünk egy Library osztályt, ami a 2. feladatban megoldott könyveket tartalmazza.
Egy Library példány tárolja osztály szinten is (overall_books), és példány szinten is (self.books) a könyveket a nyilvántartás miatt.
Hozzáadáskor ellenőrizd, hogy könyv kerüljön a Library-ba (isinstance).
A __len__ függvény egy adott példányban elhelyezett könyvek számát adja vissza.
A kiíratásakor a könyvek lapszám szerint sorrendben jelenjenek meg.
Kivételekkel kezeld le az esetleges hibákat!
A ###-ok helyett írd meg a hiányzó kódrészeket!

```
class Library:
    overall_books = []            #osztályváltozó
    def __init__(self):
        self.books = []
    def add_book(self, book):   ###
    def __len__(self):     ###
    def __str__(self):     ###
```

Library osztály

```
class Library:
   overall_books = []              #osztályváltozó
   def __init__(self):
      self.books = []
   def add_book(self, book):
      if not isinstance(book, Book):
         raise TypeError("Csak Book típusú objektumok
tárolhatóak a könyvtárban!")
      self.books.append(book)
      Library.overall_books.append(book)
   def __len__(self):
      return len(self.books)
   def __str__(self):
      sorted_books = sorted(self.books)        #page szerint
      return "\n".join(str(book) for book in sorted_books)
```

```
library1 = Library()
library1.add_book(book4)   #csökkenő oldalszámmal!
library1.add_book(book1)
library1.add_book(book2)
#library1.add_book(book3)
print(library1)
print(library1.__len__())

book_a = Book("Piroska és a farkas", "Grimm", 25)
book_b = Book("A kis hableány", "Andersen", 32)

library2 = Library()
library2.add_book(book_a)
library2.add_book(book_b)
#print(library1.overall_books)
for book in Library.overall_books:
   print(book)
print(len(Library.overall_books))
```

```
The Hobbit - J.R.R. Tolkien (250 oldal)
Harry Potter - J.K. Rowling (300 oldal)
The Lord of the Rings - J.R.R. Tolkien (500 oldal)
3
Harry Potter - J.K. Rowling (300 oldal)
The Hobbit - J.R.R. Tolkien (250 oldal)
The Lord of the Rings - J.R.R. Tolkien (500 oldal)
Piroska és a farkas - Grimm fivérek (25 oldal)
A kis hableány - Hans Christian Andersen (32 oldal)
5
```

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00